КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА
Факультет комп'ютерних наук та кібернетики
Кафедра теорії та технології програмування

Звіт з лабораторної роботи № 1
з дисципліни «Хмарні обчислення»
на тему «Паралельна обробка файлів (підрахунок частоти слів) із застосуванням системи ПАРКС у хмарному середовищі»

Виконав студент 4-го курсу
групи ТТП-42
Тимофій ЛЕЙЦИШИН

Київ — 2026

ЗМІСТ

1. Тема, мета та завдання 3
2. Платформа та хмарне середовище 3
3. Ідея алгоритму 4
4. Реалізація 5
5. Проблеми збирання та їх виправлення 5
6. Налаштування хмарного середовища 7
7. Кластер master, worker-1..4 8
8. Результати 9
9. Висновки 12

1. ТЕМА, МЕТА ТА ЗАВДАННЯ

Тема: паралельна обробка файлів (підрахунок частоти слів) із застосуванням системи ПАРКС у хмарному середовищі (варіант 4.1).

Мета: реалізувати паралельний алгоритм підрахунку частоти слів у системі ПАРКС, розгорнути систему у хмарному середовищі та оцінити залежність часу виконання від кількості воркерів.

Завдання: обрати платформу ПАРКС і реалізувати на ній алгоритм; розгорнути систему в хмарі з вузлом master і воркерами worker-1..4; виміряти час послідовного і паралельного алгоритмів для N = 1, …, 4; навести опис алгоритму, проблеми збирання та їх виправлення, інструкцію з налаштування хмарного середовища, таблицю та графік результатів.

2. ПЛАТФОРМА ТА ХМАРНЕ СЕРЕДОВИЩЕ

Використано **parcs8** – реалізацію ПАРКС на .NET 10 і Kubernetes. Вона зберігає класичну модель ПАРКС (точки, канали, рекурсивне створення точок), має спільне сховище, доступне всім точкам, і статичний пул наперед запущених демонів, у якому створення точки займає мілісекунди. Інші платформи з методичних рекомендацій відкинуто: PARCS-Python використовує Python 2.7 і модель «ведучий – робочі вузли» без точок і каналів, реалізація на Docker Swarm запускає окремий контейнер на кожну точку, PARCS.NET залежить від застарілих Windows-образів.

Хмарне середовище – **Google Kubernetes Engine** (регіон europe-central2) у межах пробного періоду Google Cloud. Кластер складається з вузла master (хост ПАРКС, портал, PostgreSQL, NFS-сервер спільного сховища) і чотирьох робочих вузлів worker-1..4 з одним демоном parcs8 на кожному; усі вузли – віртуальні машини e2-standard-2 (2 vCPU, 8 ГБ).

3. ІДЕЯ АЛГОРИТМУ

Слово – максимальна послідовність ASCII-літер і цифр, приведена до нижнього регістру. Послідовний алгоритм один раз проходить файл і накопичує частоти в хеш-таблиці. Паралельний алгоритм розпаралелює обробку за даними:

1) головна точка створює N робочих точок і канали до них (один раз на завдання);
2) файл ділиться на N суміжних діапазонів байтів рівної довжини, для чого достатньо знати його розмір;
3) кожен воркер отримує через канал лише шлях до файлу та межі свого діапазону, сам читає діапазон зі спільного сховища і рахує слова, що *починаються* в ньому;
4) воркери повертають часткові таблиці частот, головна точка додає їх до найбільшої з них і записує результат.

Правило «слово належить діапазону, в якому розташований його перший байт» забезпечує коректність за будь-якого розбиття: воркер пропускає слово, що почалося в попередньому діапазоні, і дочитує до кінця слово, що виходить за межу його діапазону. Тому кожне слово зараховується рівно один раз, а через канали передаються лише описи завдань і результати, а не вхідні дані.

4. РЕАЛІЗАЦІЯ

Код підрахунку винесено в спільну бібліотеку, яка компілюється і в модуль ПАРКС, і в послідовну програму, тож обидва алгоритми використовують однаковий код. Нижче наведено робочий модуль.

Робочий модуль ПАРКС: обробка діапазонів до сигналу завершення

```
public sealed class WorkerModule : IModule
{
    public async Task RunAsync(IModuleInfo moduleInfo,
                               CancellationToken cancellationToken = default)
    {
        while (!cancellationToken.IsCancellationRequested)
        {
            var request = await moduleInfo.Parent
                .ReadObjectAsync<WorkRequest>();
            if (request.Stop) break;
            var counts = WordCounting.NewCounts();
            var stopwatch = Stopwatch.StartNew();
            var range = new ByteRange(request.Begin, request.End);
            var stats = WordCounting.CountRange(request.Path, range, counts);
            stopwatch.Stop();
            await moduleInfo.Parent.WriteObjectAsync(new WorkResult(
                request.Round, Environment.MachineName, stats.BytesRead,
                stats.Words, stopwatch.Elapsed.TotalMilliseconds, counts));
        }
    }
}
```

Час вимірюється всередині модуля. Кожне завдання створює точки один раз і виконує над ними 1 розминочний і 5 вимірюваних раундів; у результатах наведено медіану. Створення точок (5–28 мс) вимірюється окремо. Послідовна програма виконується на вузлі worker-1 з тими самими обмеженнями ресурсів, що й демон (1 vCPU, 2 ГіБ), над тим самим файлом зі спільного сховища.

5. ПРОБЛЕМИ ЗБИРАННЯ ТА ЇХ ВИПРАВЛЕННЯ

Основні проблеми, пов’язані із застарілими або неопублікованими залежностями та помилками конфігурації parcs8, і способи їх виправлення наведено в табл. 1.

Таблиця 1 – Проблеми збирання і розгортання parcs8 та їх виправлення

Група	Проблема	Виправлення
Залежності	пакет `Parcs.Net` 10.1.0 не опубліковано на nuget.org; `dotnet pack` падає з NU5026 (`GeneratePackageOnBuild`)	пакет збирається з вихідного коду в локальний репозиторій пакетів під час Docker-збирання
Образи	маніфест для GCP посилається на образи в проєкті автора; опубліковані образи не містять виправлень	збирання з вихідного коду сервісом Cloud Build у власний Artifact Registry
Маніфести	PVC не прив'язуються до PV, неіснуюча заявка Elasticsearch, порт 8080 замість 80, `postgres:latest` = 18	власні маніфести kustomize, `ASPNETCORE_URLS`, `postgres:16`
Elasticsearch	жорстка залежність лише для журналювання; портал падає без неї	латка `patches/0001`: Elasticsearch вимикається порожньою адресою
Запуск хоста	міграція БД без повторних спроб: хост не стартує, якщо PostgreSQL ще не готовий	проби `/health`, Kubernetes перезапускає хост
Хмара (GKE)	немає сховища ReadWriteMany; заявка більша за диск NFS; нестача пам'яті воркера і хоста на наборі з 6,9 млн різних слів	NFS-Ganesha, заявка 10 ГіБ, ліміти 2 ГіБ (демон) і 5 ГіБ (хост), злиття у найбільший частковий результат

6. НАЛАШТУВАННЯ ХМАРНОГО СЕРЕДОВИЩА

Розгортання виконується утилітами `gcloud`, `kubectl` і `helm` у наведеній нижче послідовності. Ідентифікатор проєкту – `parcs-wordcount-tl`, регіон – europe-central2; маніфести Kubernetes (`deploy/`), конфігурацію Cloud Build (`docker/cloudbuild.yaml`) і значення Helm-чарту наведено в репозиторії проєкту.

Створення проєкту, образів і кластера

```
P=parcs-wordcount-tl; R=europe-central2; Z=europe-central2-a
REG=$R-docker.pkg.dev/$P/parcs
SA=<номер-проєкту>-compute@developer.gserviceaccount.com

# 1. Проєкт, білінг, API, роль для Cloud Build
gcloud auth login
gcloud projects create $P
gcloud billing projects link $P --billing-account=<ID>
gcloud config set project $P
gcloud services enable compute.googleapis.com container.googleapis.com \
    artifactregistry.googleapis.com cloudbuild.googleapis.com
gcloud projects add-iam-policy-binding $P \
    --role=roles/cloudbuild.builds.builder --member=serviceAccount:$SA

# 2. Образи daemon, host, portal, seq: Cloud Build -> Artifact Registry
gcloud artifacts repositories create parcs --location=$R \
    --repository-format=docker
gcloud builds submit <каталог-збирання> --config=docker/cloudbuild.yaml \
    --region=$R --substitutions=_REGISTRY=$REG,_TAG=v1

# 3. Кластер: вузол master і пул із чотирьох робочих вузлів
gcloud container clusters create parcs --zone=$Z --num-nodes=1 \
    --machine-type=e2-standard-2 --node-labels=parcs-role=master \
    --disk-type=pd-standard --disk-size=30 \
    --logging=NONE --monitoring=NONE
gcloud container node-pools create workers --cluster=parcs --zone=$Z \
    --num-nodes=4 --machine-type=e2-standard-2 \
    --node-labels=parcs-role=worker --disk-type=pd-standard --disk-size=30
gcloud container clusters get-credentials parcs --zone=$Z
kubectl label node <і-й робочий вузол> parcs-worker=worker-<i>  # i = 1..4
kubectl get nodes -L parcs-role,parcs-worker
```

Розгортання parcs8, запуск завдань і видалення ресурсів

```
# 4. Спільне сховище (NFS-Ganesha) і parcs8 у статичному режимі
G=kubernetes-sigs.github.io; C=nfs-ganesha-server-and-external-provisioner
helm repo add nfs-ganesha https://$G/$C/
helm install nfs-server nfs-ganesha/nfs-server-provisioner \
    --version 1.8.0 -f deploy/gcp/nfs-values.yaml
kubectl apply -k deploy/gcp        # демони, хост, портал, PostgreSQL
kubectl get pods -o wide

# 5. Набори даних у спільному сховищі
kubectl create configmap datagen --from-file=scripts/datagen.py
kubectl apply -f deploy/jobs/datagen-job.yaml

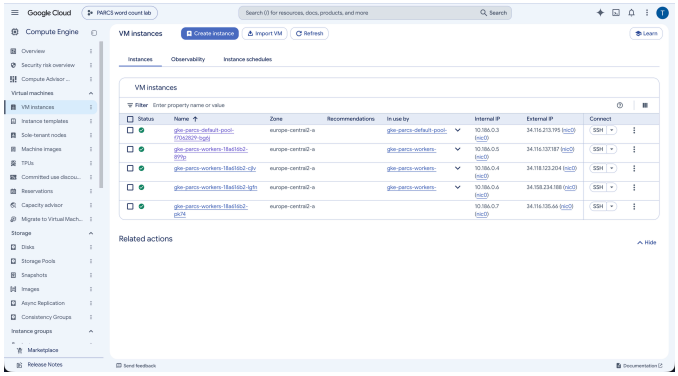
# 6. Доступ до API хоста; модуль, завдання і запуск (або в порталі)
kubectl port-forward svc/parcs-hostapi 15080:80
H=http://127.0.0.1:15080/api
curl -F Name=wordcount -F BinaryFiles=@Parcs.Modules.WordCount.dll \
     -F BinaryFiles=@Parcs.Modules.WordCount.deps.json $H/Modules
curl -F ModuleId=1 -F AssemblyName=Parcs.Modules.WordCount \
     -F ClassName=Parcs.Modules.WordCount.MainModule \
     -F InputFiles=@input.txt $H/Jobs
curl -H 'Content-Type: application/json' $H/SynchronousJobRuns -d '{
  "jobId": 1, "arguments": {"PointsNumber": "4", "Repetitions": "5",
  "Warmup": "1", "InputPath": "datasets/natural_1000mb.txt"}}'
curl -o job-1.zip $H/JobOutputs/1    # timings.json, frequencies.txt

# 7. Послідовний алгоритм на вузлі worker-1
kubectl apply -f deploy/jobs/seq-job.yaml
kubectl logs -f job/wordcount-seq

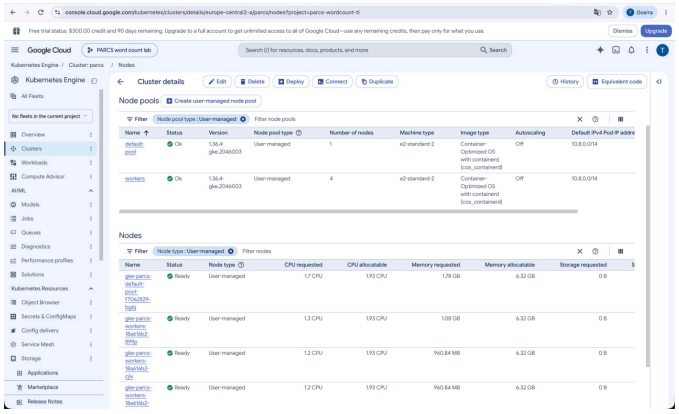
# 8. Видалення ресурсів
kubectl delete -k deploy/gcp && helm uninstall nfs-server
kubectl delete pvc --all
gcloud container clusters delete parcs --zone=$Z
gcloud artifacts repositories delete parcs --location=$R
```

7. КЛАСТЕР MASTER, WORKER-1..4

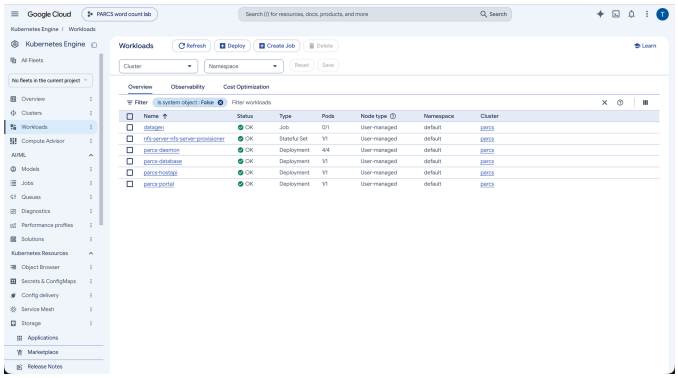
Розгорнутий кластер показано на рис. 1: віртуальні машини вузла master (пул `default-pool`) і робочих вузлів worker-1..4 (пул `workers`), пули вузлів GKE, робочі навантаження (чотири поди `parcs-daemon`, по одному на робочий вузол, хост, портал, PostgreSQL і NFS-сервер) та завдання модуля `wordcount` у порталі parcs8. Повний цикл розгортання, вимірювань і видалення ресурсів тривав близько години і використав менше одного долара пробних кредитів.



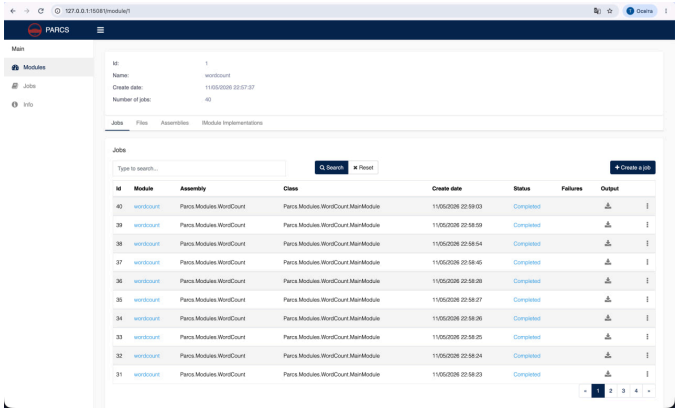
а) віртуальні машини master і worker-1..4



б) пули і вузли кластера GKE



в) робочі навантаження кластера



г) завдання модуля wordcount у порталі

Рисунок 1 – Кластер parcs8 у Google Kubernetes Engine

8. РЕЗУЛЬТАТИ

Використано набори *natural* (текст зі словником із 96 слів; 100, 500 і 1000 МБ) і *highcard* (100 МБ, 6,9 млн різних слів серед 7,8 млн). Перед вимірюваннями вихід паралельного алгоритму порівняно побайтово з еталонною реалізацією на 9 наборах, зокрема навмисно складних для правила меж, для N = 1, …, 4: усі 36 результатів збіглися з еталоном.

Медіани часу, прискорення $S(N) = T_1/T_N$ та ефективність $E(N) = S(N)/N$ наведено в табл. 2, графіки – на рис. 2 і 3.

Таблиця 2 – Медіана часу раунду, прискорення та ефективність відносно послідовного алгоритму

Набір	Показник	Послід.	N = 1	N = 2	N = 3	N = 4
natural, 100 МБ	час, с	1,35	1,36	0,74	0,49	0,42
	прискорення	1,00	1,00	1,82	2,79	3,26
	ефективність	1,00	1,00	0,91	0,93	0,82
natural, 500 МБ	час, с	6,83	6,52	3,62	2,46	1,82
	прискорення	1,00	1,05	1,89	2,78	3,75
	ефективність	1,00	1,05	0,94	0,93	0,94
natural, 1000 МБ	час, с	13,66	13,92	7,01	4,74	3,52
	прискорення	1,00	0,98	1,95	2,88	3,88
	ефективність	1,00	0,98	0,98	0,96	0,97
highcard, 100 МБ	час, с	3,74	9,27	8,39	7,28	7,61
	прискорення	1,00	0,40	0,45	0,51	0,49
	ефективність	1,00	0,40	0,22	0,17	0,12

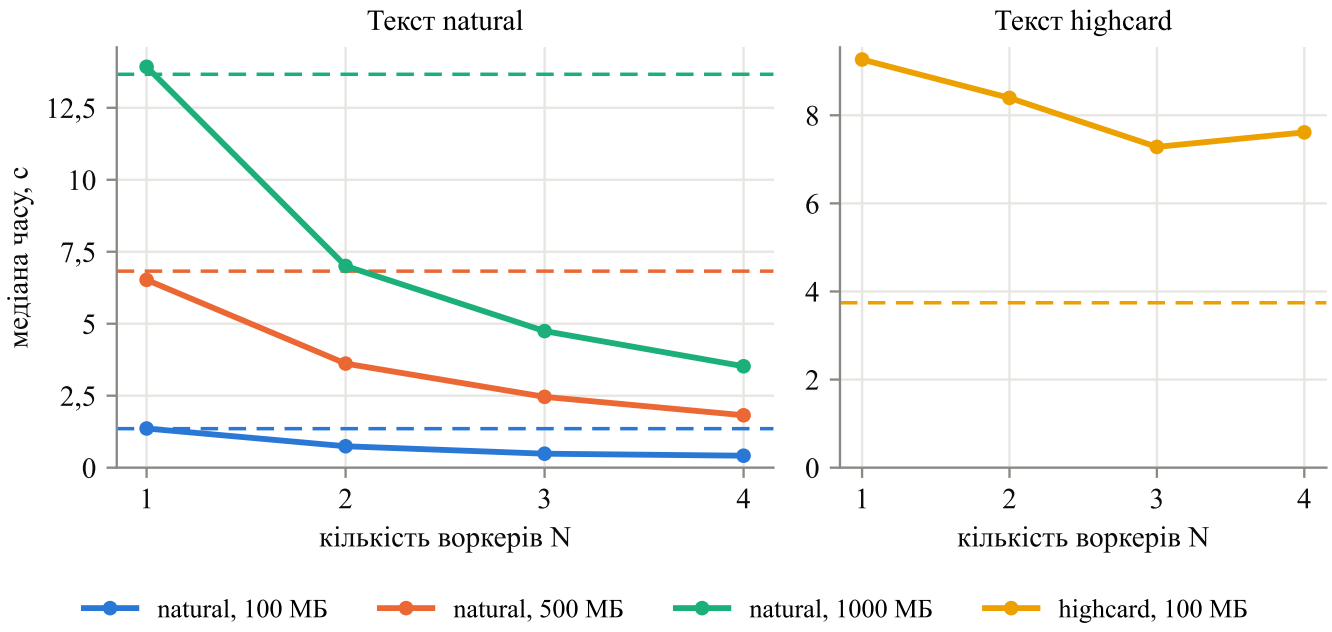


Рисунок 2 – Залежність часу виконання від кількості воркерів; штрихові лінії – послідовний алгоритм

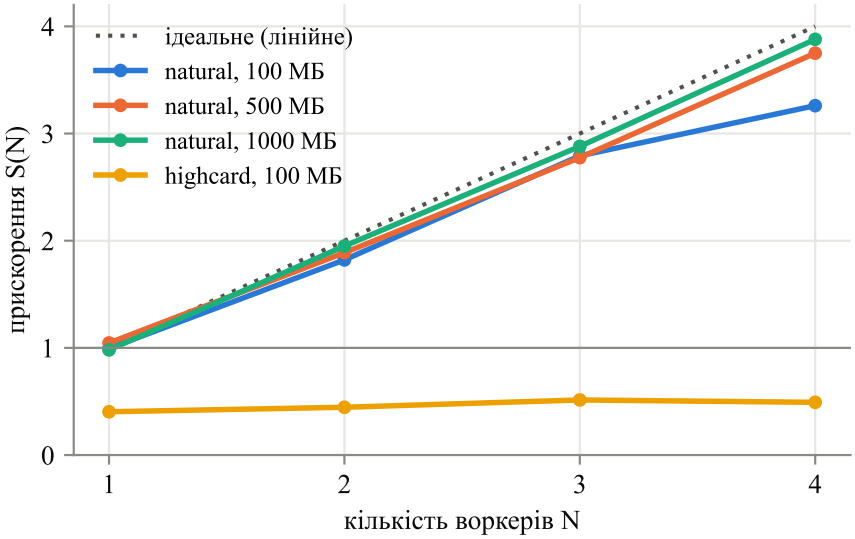


Рисунок 3 – Прискорення відносно послідовного алгоритму

На тексті natural прискорення близьке до лінійного і зростає з розміром файлу: на 4 воркерах 3,26 (100 МБ), 3,75 (500 МБ) і 3,88 (1000 МБ, ефективність 0,97). Часткові таблиці містять лише 96 записів, тому обмін і злиття займають кілька мілісекунд, а час раунду майже повністю визначається скануванням, яке рівномірно ділиться між воркерами. На тексті highcard паралельний алгоритм повільніший за послідовний (прискорення 0,40–0,51), хоча час сканування найповільнішого воркера зменшується з 3,80 до 0,95 с (рис. 4): пересилання, десеріалізація та злиття таблиць із мільйонами записів займають 5–7 с на одному вузлі й не зменшуються з ростом N.

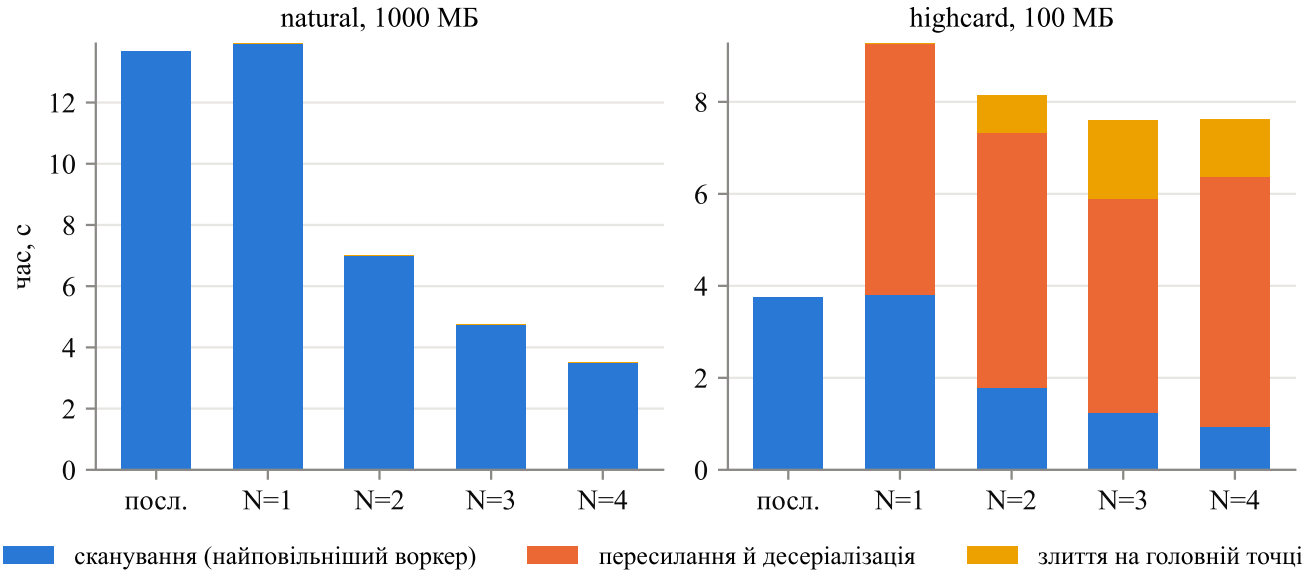


Рисунок 4 – Фази раунду: сканування найповільнішого воркера, пересилання й десеріалізація часткових таблиць, злиття на головній точці

9. ВИСНОВКИ

1) Реалізовано паралельний підрахунок частоти слів у системі ПАРКС (parcs8): воркери самостійно читають свої діапазони файлу зі спільного сховища, а головна точка об’єднує часткові результати. Коректність підтверджено побайтовим збігом з еталоном на 9 наборах для N = 1, …, 4.
2) parcs8 розгорнуто в Google Kubernetes Engine з вузлом master і чотирма робочими вузлами. Для цього усунуто проблеми збирання (неопублікований пакет `Parcs.Net`, образи, прив’язані до проєкту автора) і конфігурації (маніфести, залежність від Elasticsearch, ресурси в хмарі).
3) Для тексту з невеликим словником отримано прискорення, близьке до лінійного, – до 3,88 на 4 воркерах; ефективність зростає з обсягом даних, оскільки фіксовані витрати раунду стають відносно меншими.
4) Для тексту з дуже великим словником паралельна обробка неефективна: обмін і централізоване злиття часткових результатів займають більше часу, ніж обчислення (закон Амдала). Отже, ефективність розподіленої обробки визначається співвідношенням обсягу обчислень і обсягу переданих даних, а не лише кількістю вузлів; для таких даних потрібне розподілене злиття за хешем слова із записом результату частинами.

Код проєкту: github.com/leitsyshyn/cloudcompute